

MuJoCo & Simulation Tools (Gazebo, MuJoCo, Velocity & Torque Control)

Introduction to Robotics Lab

Konstantinos Asimakopoulos k_asimakopoulos@ac.upatras.gr
Lampros Printzios printzios_lampros@ac.upatras.gr

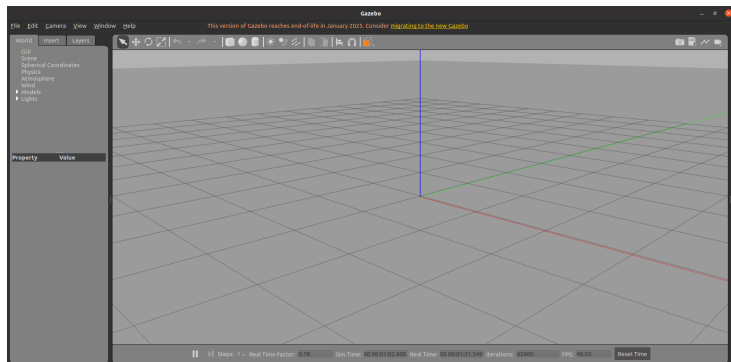
Department of Electrical and Computer Engineering
University of Patras

December 9, 2025

Table of Contents

- 1 Gazebo
 - What is Gazebo?
- 2 MuJoCo
 - What is MuJoCo?
- 3 Basic Control Schemes for Manipulators
 - Basic Types of Control
 - Velocity Control in the Operational/Task Space
 - Torque/Force Control in the Joint Space
- 4 Controlling Manipulators inside MuJoCo Simulator
 - Kuka LBR iiwa 14
 - Desired Trajectories for Tracking
 - Part 1: Velocity Control in Task Space for Kuka
 - Part 2: Torque Control in Joint Space for Kuka
- 5 References

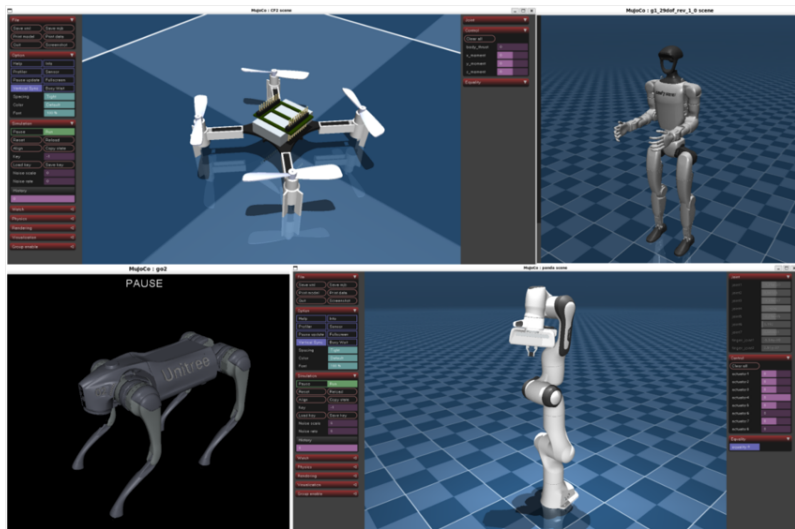
Gazebo is a **robot simulator**. The difference between it and RViz was summed up by Morgan Quigley (one of the original ROS developers) in his book "Programming Robots with ROS": "... *rviz shows you what the robot thinks is happening, while Gazebo shows you what is really happening ...*".



Gazebo starting Environment

Gazebo - What is Gazebo?

- ① **Gazebo** is a **robot simulation tool** widely used with ROS. It provides realistic physics, sensors, and environments for testing and developing robotic applications without requiring a physical robot.
- ② Key Features and Uses:
 - **Physics-based Simulation**: simulates rigid body dynamics, collisions, and joint constraints
 - **Sensor Simulation**: emulates cameras, LIDARs, IMUs, and other robot sensors
 - **Environment Modeling**: allows the creation of complex 3D worlds, obstacles, and terrains
 - **Integration with ROS**: easily connect with ROS/ROS2 nodes for testing algorithms in a virtual environment
- ③ While Gazebo is powerful and widely used, modern alternatives like **MuJoCo** offer ***improved speed, stability, and advanced physics capabilities***, making them better suited for certain research and control tasks.



Robots examples inside MuJoCo Simulator

MuJoCo - What is MuJoCo?

- 1 **MuJoCo** (Multi-Joint dynamics with Contact) is a **modern physics engine** for simulating robots and complex mechanical systems. It focuses on ***high-performance, accurate, and efficient simulations***.
- 2 Key Features and Uses:
 - **Fast and Stable Physics:** optimized for speed and numerical stability, even with many contacts
 - **Advanced Contact Dynamics:** handles friction, collision, and constraints more realistically
 - **Flexible Robot Modeling:** supports articulated robots, custom actuators, and sensors
 - **Integration with Learning Frameworks:** widely used in reinforcement learning and control research due to high simulation speed
- 3 Compared to Gazebo, MuJoCo offers ***faster simulations, better control over physics parameters, and more capabilities for research-oriented tasks***, making it our tool of choice for this lab.

Basic Control Schemes for Manipulators - Basic Types of Control

① Joint (Servo) Control

- **Joint Position Control:** regulates each joint's angle to achieve a desired configuration
- **Joint Velocity Control:** controls the speed of each joint for smooth motion or trajectory tracking
- **Joint Torque/Force Control:** *applies specific torque or force at joints to handle dynamic tasks or interactions*

② Operational/Task Space Control

- **Cartesian Position Control:** moves the end-effector to a desired position and orientation in 3D space
- **Cartesian Velocity Control:** *regulates the end-effector speed along a trajectory or in response to task requirements*
- **Cartesian Force/Impedance Control:** controls interaction forces with the environment, allowing compliant or stiff behavior as needed

Other control approaches: Impedance/Admittance Control, Model-Based Control (Adaptive, Robust, etc.), Learning-Based Control, Model Predictive Control (MPC)

Basic Control Schemes for Manipulators - Velocity Control in the Operational/Task Space

- ① Given a desired end-effector velocity profile $\mathcal{V}_d(t)$ in world frame, we define the task-space error $\mathcal{X}_{error}$ in world frame:

$$\mathcal{X}_{error} = \begin{bmatrix} \mathbf{p}_{des} - \mathbf{p}_{curr} \\ \frac{1}{2} \text{vee} (\mathbf{R}_{des} \mathbf{R}_{curr}^\top - \mathbf{R}_{curr} \mathbf{R}_{des}^\top) \end{bmatrix}, \quad \text{vee} \left(\begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix} \right) = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix}$$

- ② We construct the task-space PI control law for the end-effector ($\mathcal{V}$ in world frame), where $K_p, K_i \in \mathbb{R}^{6 \times 6}$ are positive diagonal gain matrices for translational and rotational components:

$$\mathcal{V}(t) = \underbrace{\mathcal{V}_d(t)}_{\text{Feedforward}} + \underbrace{K_p \mathcal{X}_{error}(t)}_{\text{Proportional Feedback}} + \underbrace{K_i \int_0^t \mathcal{X}_{error}(\tau) d\tau}_{\text{Integral Feedback}}$$

- ③ Task-space velocity $\mathcal{V}$ maps to joint velocities $\mathbf{v}$ via the pseudoinverse $J^\dagger$ of the world-frame $6 \times n_v$ Jacobian J :

$$\mathcal{V} = J\mathbf{v} \quad \implies \quad \mathbf{v}(t) = J^\dagger \mathcal{V}(t)$$

- ④ Finally, joint velocities are integrated kinematically:

$$\mathbf{q}(t + \Delta t) = \mathbf{q}(t) + \mathbf{v}(t)\Delta t$$

Basic Control Schemes for Manipulators - Torque/Force Control in the Joint Space

- Given a desired trajectory, we define the 6D task-space error $\mathcal{X}_{error}$ and its derivative $\mathcal{V}_{error}$:

$$\mathcal{X}_{error} = \begin{bmatrix} \mathbf{p}_{des} - \mathbf{p}_{curr} \\ \frac{1}{2} \text{vee} (\mathbf{R}_{des} \mathbf{R}_{curr}^T - \mathbf{R}_{curr} \mathbf{R}_{des}^T) \end{bmatrix}, \quad \mathcal{V}_{error} = \dot{\mathcal{X}}_{error} = \begin{bmatrix} \dot{\mathbf{p}}_{des} - \dot{\mathbf{p}}_{curr} \\ \boldsymbol{\omega}_{des} - \boldsymbol{\omega}_{curr} \end{bmatrix}$$

- We compute the task-space desired wrench force $\mathbf{F}_{task}$ using PD control, where $K_p, K_d \in \mathbb{R}^{6 \times 6}$ are positive diagonal gain matrices for translation and rotation; and $M_x = (J M^{-1} J^T)^{-1}$ is the task-space inertia matrix, where M is the joint-space inertia matrix and J is the $6 \times n_v$ world-frame Jacobian:

$$\mathbf{F}_{task} = M_x (K_p \mathcal{X}_{error} + K_d \mathcal{V}_{error})$$

- Task-space wrench force maps to joint torques $\boldsymbol{\tau}_{task}$ via the transpose J^T :

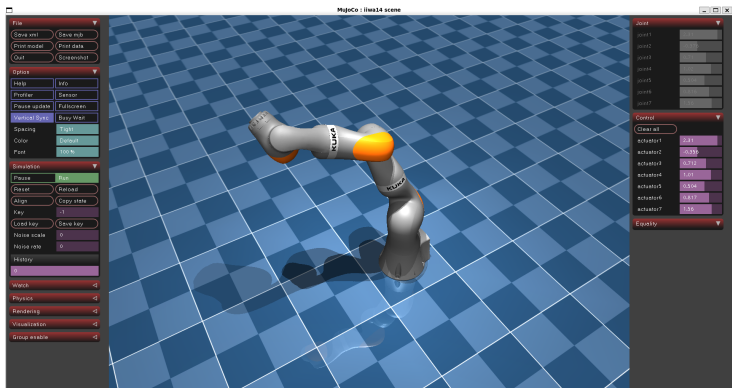
$$\boldsymbol{\tau}_{task} = J^T \mathbf{F}_{task}$$

- We also include joint-space gravity compensation $\boldsymbol{\tau}_{gravity}$ (Coriolis, centrifugal and gravitational bias forces) and add a null-space regularization torque $\boldsymbol{\tau}_{null}$. Using the dynamically consistent pseudoinverse $\bar{J} = M^{-1} J^T M_x$, the combined joint-space control torque $\boldsymbol{\tau}_{control}$ becomes:

$$\boldsymbol{\tau}_{control} = \underbrace{\boldsymbol{\tau}_{task} + \boldsymbol{\tau}_{gravity}}_{\text{Task + Gravity Compensation}} + \underbrace{\left(I - J^T \bar{J} \right) \left(K_{p,null} \left(\frac{\mathbf{q}_{min} + \mathbf{q}_{max}}{2} - \mathbf{q} \right) - K_{d,null} \dot{\mathbf{q}} \right)}_{\boldsymbol{\tau}_{null}}.$$

Controlling Manipulators inside MuJoCo Simulator - Kuka LBR iiwa 14

We are going to control the 7-DOF Kuka LBR iiwa 14 robotic manipulator inside the MuJoCo simulator!



Kuka LBR iiwa 14 at random configuration in MuJoCo

Controlling Manipulators inside MuJoCo Simulator - Desired Trajectories for Tracking

We will attempt to track a set of trajectories in 3D space using the robot's end-effector. The desired trajectories, defined mathematically below using parametric equations, are assumed to lie in a plane parallel to the xy plane ($z(t) = h$, with $\dot{z}(t) = 0$). Throughout the motion, the end-effector should maintain the same orientation as its initial default zero pose.

1 Circle:

$$\begin{aligned}x(t) &= x_0 + r \cos(\omega t) & \dot{x}(t) &= -r\omega \sin(\omega t) \\ y(t) &= y_0 + r \sin(\omega t) & \dot{y}(t) &= r\omega \cos(\omega t)\end{aligned}$$

2 Lissajous Curves:

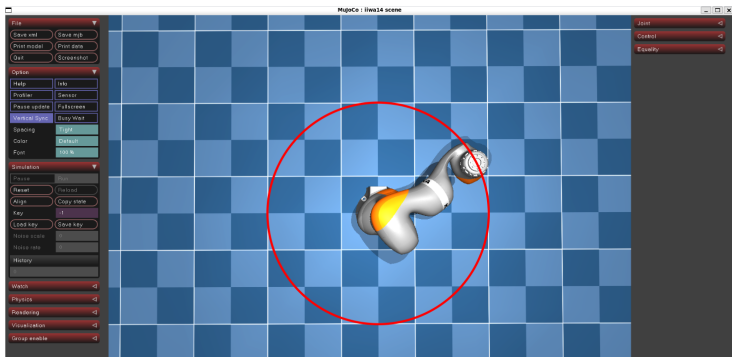
$$\begin{aligned}x(t) &= r \sin(\omega t) & \dot{x}(t) &= r\omega \cos(\omega t) \\ y(t) &= r \sin(n\omega t) & \dot{y}(t) &= n r \omega \cos(n\omega t)\end{aligned}$$

3 Eight Figures:

$$\begin{aligned}x(t) &= b \frac{\cos(\omega t)}{1 + \sin^2(\omega t)} & \dot{x}(t) &= b \frac{-2\omega \sin(\omega t) (\cos(2\omega t) + 5)}{(\cos(2\omega t) - 3)^2} \\ y(t) &= c \frac{\sqrt{2} \sin(2\omega t)}{1 + \sin^2(\omega t)} & \dot{y}(t) &= c \frac{4\sqrt{2}\omega (3 \cos(2\omega t) - 1)}{(\cos(2\omega t) - 3)^2}\end{aligned}$$

Controlling Manipulators inside MuJoCo Simulator - Part 1: Velocity Control in Task Space for Kuka

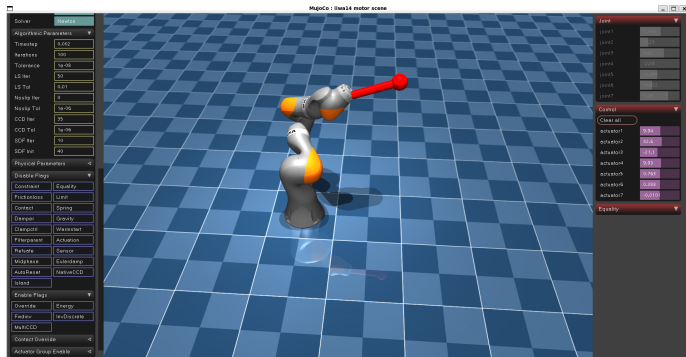
First, we will apply **Velocity Control in 3D Task Space**. At this stage, simulation mode remains disabled, so the system dynamics are not active (meaning that the robot's mass, inertia, etc. do not play any role on its movements). This step is essentially a kinematic visualization task.



Kuka tracking a circular trajectory

Controlling Manipulators inside MuJoCo Simulator - Part 2: Torque Control in Joint Space for Kuka

For **Torque Control in Joint Space**, we activate the system dynamics. As the robot tracks the desired trajectory, we can disturb it by applying forces/torques at various locations. This lets us observe how the robot responds to external interactions and how it attempts to return to the prescribed trajectory.



Interacting with Kuka dynamics

- **MuJoCo documentation**
- **A collection of high-quality models for the MuJoCo physics engine, curated by Google DeepMind (Mujoco Menagerie)**
- **Modern Robotics: Mechanics, Planning, and Control; Kevin M. Lynch and Frank C. Park**
- **Kuka LBR iiwa 14 manual**